
Algorithms

Five procedures, each one traced by hand on a six-element array.

Today

AGENDA

- 01 What an algorithm is, and how we write one down
- 02 Finding the maximum
- 03 Linear search and binary search
- 04 Bubble sort and insertion sort
- 05 Counting the work each one does

algorithm

noun

Word used by programmers when they do not want to explain what they did.

What Is an Algorithm?

DEFINITION

DEFINITION

An **algorithm** is a finite sequence of precise instructions for performing a computation or for solving a problem.

Finite — it has to stop.

Precise — no step may require judgement.

We write algorithms in **pseudocode** : precise about the steps, silent about the syntax. Arrays here are indexed from 1, not 0.

Finding the Maximum

MAXIMUM

THE IDEA

Keep a running **champion** . Walk the list once; whenever you meet something bigger, it takes the title.

INPUT REQUIRED

STRUCTURE	Array or list $a_1 \dots a_n$, read once, front to back
ORDERING	None. Any arrangement works.
ELEMENTS	Integers — anything you can compare with $<$
RETURNS	The largest value (not its position)

THINK OF IT AS

A knockout tournament where the winner stays on. Every contestant fights exactly once, so the whole thing takes one pass.

- 1 Take the first number and call it the champion.
- 2 Move to the next number in the list.
- 3 If it beats the champion, it becomes the new champion. Otherwise leave the champion alone.
- 4 Keep going until the list runs out. The champion is the largest value.

Trace: Maximum of 7 2 1 4 5 9

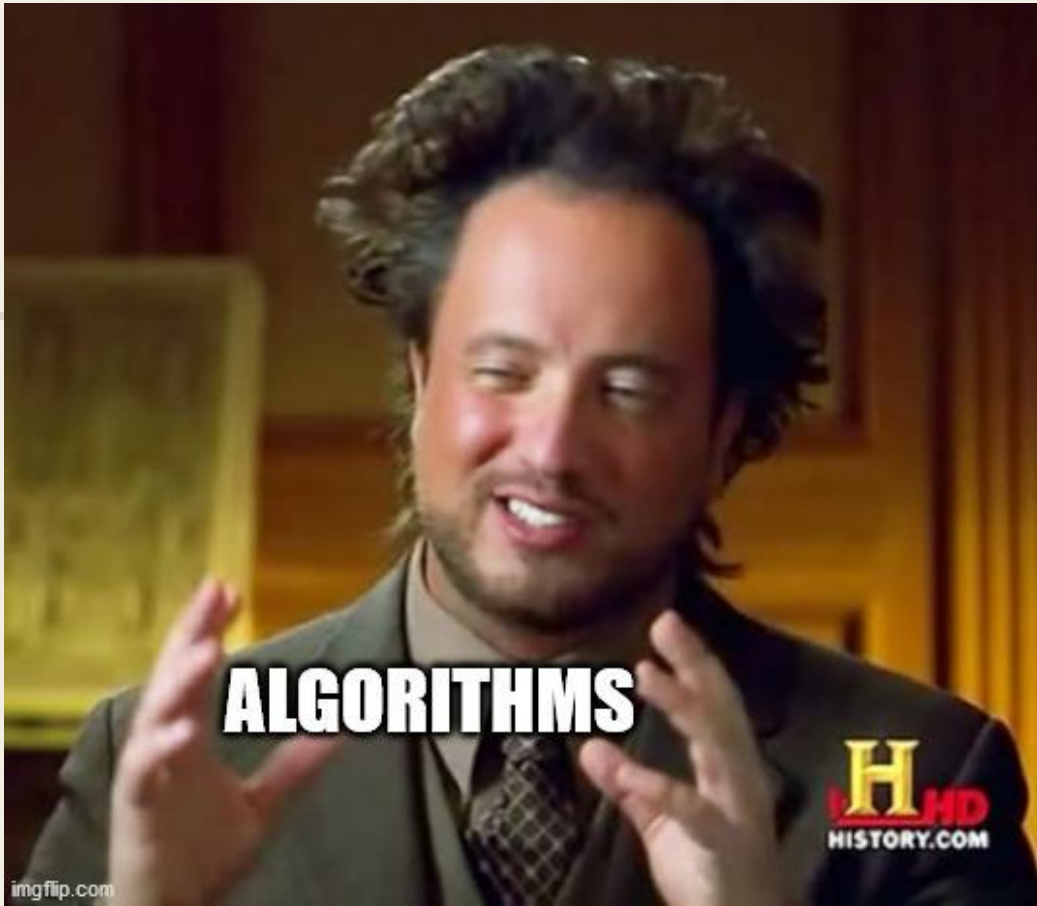
MAXIMUM

i	a _i	max < a _i ?	max after
—	—	init	7
2	2	7 < 2 no	7
3	1	7 < 1 no	7
4	4	7 < 4 no	7
5	5	7 < 5 no	7
6	9	7 < 9 YES	9

return 9

Six elements, five comparisons. max was reassigned once.

The comparison count never changes — it is $n - 1$ on every input. Only the number of reassignments varies, and nobody counts those.



The Maximum, Formally

MAXIMUM

```
MAX( $a_1, \dots, a_n$ : integers)
```

```
1: max  $\leftarrow a_1$ 
2: for  $i \leftarrow 2$  to  $n$  do
3:   if max  $< a_i$  then
4:     max  $\leftarrow a_i$ 
5:   end if
6: end for
7: return max
```

Seven lines. Line 1 sets the champion; lines 3–4 are the only place it ever changes.

The loop starts at $i = 2$, not 1. Starting at 1 compares a_1 against itself — correct, but a wasted step.

Total comparisons: $n - 1$

“PRECISE INSTRUCTIONS”

“Season to taste” is not an algorithm.

Every step has to be executable by something that cannot think. If a line needs judgement, it is not a step yet.



Searching Problems

SEARCHING

THE PROBLEM

Given a list $a_1, \dots, a_n$ and a value x , find the position of x in the list, or report that it is not there. We return 0 for “not there,” which is why our arrays start at 1.

LINEAR SEARCH

Works on **any** list.

No ordering required. It just looks at everything.

BINARY SEARCH

Requires a **sorted** list.

That requirement is exactly what buys the speed.

Linear Search

THE IDEA

Look at every position in turn until you find what you want — or until you run out of list.

INPUT REQUIRED

STRUCTURE	Array $a_1 \dots a_n$, indexed from 1
ORDERING	None required — this is its advantage
ELEMENTS	Distinct integers, plus the target x
RETURNS	Position of x , or 0 if absent

THINK OF IT AS

Looking for a friend in a crowd by checking faces one at a time. Nobody is standing in any particular order, and it does not matter.

- 1 Start at the first position.
- 2 Compare the value there with the one you are looking for.
- 3 If they match, stop. That position is the answer.
- 4 If not, step one place right and compare again.
- 5 If you step off the end of the list, the value is not there. Report 0.

Trace: Linear Search

SEARCHING

`a = 7 2 1 4 5 9` unsorted — linear search does not care

`x = 4` (present)

i	$i \leq 6$?	$x \neq a_i$?	action
1	yes	$4 \neq 7$	$i \leftarrow 2$
2	yes	$4 \neq 2$	$i \leftarrow 3$
3	yes	$4 \neq 1$	$i \leftarrow 4$
4	yes	$4 = 4$	exit loop

`i = 4 ≤ 6 → return 4`

4 comparisons against array entries.

A failed search is the expensive one: it always costs all n comparisons. Notice the last row on the right — a_7 is never read, because $i \leq n$ failed first.

`x = 6` (absent)

i	$i \leq 6$?	$x \neq a_i$?	action
1	yes	$6 \neq 7$	$i \leftarrow 2$
2-4	yes	$6 \neq 2, 1, 4$	$i \leftarrow 5$
5	yes	$6 \neq 5$	$i \leftarrow 6$
6	yes	$6 \neq 9$	$i \leftarrow 7$
7	NO	not evaluated	exit loop

`i = 7 > 6 → return 0`

6 comparisons — the full array.

Linear Search, Formally

```
LINEARSEARCH( $x$ ;  $a_1, \dots, a_n$ : distinct integers)
```

```
1:  $i \leftarrow 1$ 
2: while ( $i \leq n$  and  $x \neq a_i$ ) do
3:    $i \leftarrow i + 1$ 
4: end while
5: if ( $i \leq n$ ) then
6:   location  $\leftarrow i$ 
7: else
8:   location  $\leftarrow 0$ 
9: end if
10: return location
```

The order inside the `while` condition matters.
Check $i \leq n$ *first*, or you read past the end of the array.

That is short-circuit evaluation of $\wedge$, straight from Lecture 1.

After the loop, i alone tells you which case you are in: $i \leq n$ means found.

Binary Search

THE IDEA

Track the stretch of list that could still hold your value. Check its middle, then throw away the half that cannot contain it. Repeat.

INPUT REQUIRED

STRUCTURE	Array with direct index access — you must reach a_m in one step, so a linked list will not do
ORDERING	Must be sorted increasing. Non-negotiable.
ELEMENTS	Distinct integers, plus the target x
RETURNS	Position of x , or 0 if absent

THINK OF IT AS

Guessing a number between 1 and 100 while being told “higher” or “lower.” You say 50 first. That instinct is binary search — and it needs the list sorted, or “higher” and “lower” mean nothing.

- 1 Start with the whole list as your search range.
- 2 Look at the entry in the middle of the range.
- 3 If your value is larger, discard the middle entry and everything to its left.
- 4 Otherwise discard everything to the right of the middle.
- 5 Repeat until one candidate is left, then check whether it is actually your value.

Trace: Binary Search for 19

SEARCHING

index	1	2	3	4	5	6	7	8
value	2	5	7	11	14	19	23	30

pass	i	j	$m = \lfloor (i+j)/2 \rfloor$	a_m	$19 > a_m ?$	update	candidates left
1	1	8	4	11	yes	$i \leftarrow 5$	$8 \rightarrow 4$
2	5	8	6	19	no	$j \leftarrow 6$	$4 \rightarrow 2$
3	5	6	5	14	yes	$i \leftarrow 6$	$2 \rightarrow 1$
exit	6	6	$i < j$ is false			$a_6 = 19 \checkmark$	return 6

Three passes. $\lceil \log_2 8 \rceil = 3$ — exactly. Linear search would have needed six comparisons to reach position 6.

Trace: Searching for 6, Which Is Absent

SEARCHING

pass	i	j	m	a _m	6 > a _m ?	update
1	1	8	4	11	no	j ← 4
2	1	4	2	5	yes	i ← 3
3	3	4	3	7	no	j ← 3
exit	3	3	a ₃ = 7 ≠ 6 → return 0			

The interval still collapses to a single candidate. Line 11 is what catches the miss.

Binary search never notices absence early. It runs all $\lceil \log_2 n \rceil$ passes and finds out at the last check.

Compare linear search, where a miss costs the whole array. Here a miss costs the same three passes as a hit.

Binary Search, Formally

SEARCHING

```
BINARYSEARCH(x; a1, ..., an: increasing integers)
```

```
1: i ← 1 /* left end */
2: j ← n /* right end */
3: while (i < j) do
4:   m ← ⌊(i + j)/2⌋
5:   if (x > am) then
6:     i ← m + 1
7:   else
8:     j ← m
9:   end if
10: end while
11: if x = ai then location ← i
12: else location ← 0
13: return location
```

Two pointers bracket the interval that could still contain x . Each pass halves it.

WATCH LINE 8

$j \leftarrow m$, not $m - 1$. When $x \leq a_m$, the midpoint itself could still be the answer.

The loop exits when $i = j$ — one candidate left. Line 11 is the only place we test for equality.

Linear vs Binary, in Numbers

SEARCHING

n	linear (worst)	binary
8	8	3
1,000	1,000	10
1,000,000	1,000,000	20
10^9	10^9	30

Double the array and linear search doubles.

Binary search adds **one** comparison.

The catch: the list has to be sorted first, and sorting costs more than either search. If you search once, sort nothing. If you search often, sort once.

Naming those two growth rates precisely is the whole of Lecture 7.

The interviewer watching me fumble a binary search on the whiteboard after I bragged about building a P2P file sharing app from scratch on my resume



PRECONDITIONS ARE NOT SUGGESTIONS

Binary search on an unsorted array doesn't crash.

It returns 0 and looks like it worked. Silent wrongness is the worst kind.

Sorting

SORTING

THE PROBLEM

Given a list of elements and a way to order them, **sorting** is rearranging the list so that the elements appear in increasing order.

7 2 1 4 5 9 → 1 2 4 5 7 9

Two algorithms, the same input, and very different amounts of work. We will count both.

Bubble Sort

THE IDEA

Walk the list comparing each pair of **neighbours** , swapping any that are the wrong way round. Repeat until nothing is out of order.

INPUT REQUIRED

STRUCTURE	Array $a_1 \dots a_n$ with $n \geq 2$, rearranged in place
ORDERING	None. Any arrangement is valid input.
ELEMENTS	Real numbers — anything totally ordered
RETURNS	Nothing. The array itself is left sorted.

THINK OF IT AS

Large bubbles rising through water. Each pass floats the biggest remaining value up to the surface, one swap at a time.

- 1 Compare the first two entries. If the left one is larger, swap them.
- 2 Move one place right and compare that pair. Swap if needed.
- 3 Carry on to the end of the list. The largest value has now been dragged to the far right.
- 4 That last entry is finished, so walk again over everything before it.
- 5 Each walk finishes one more entry. After $n-1$ walks the list is sorted.

Trace: Bubble Sort, Pass 1

j	compare	swap?	list after
—	start	—	7 2 1 4 5 9
1	7 > 2	yes	2 7 1 4 5 9
2	7 > 1	yes	2 1 7 4 5 9
3	7 > 4	yes	2 1 4 7 5 9
4	7 > 5	yes	2 1 4 5 7 9
5	7 > 9	no	2 1 4 5 7 9

Five comparisons, four swaps. The 7 travels right one position at a time until it meets the 9.

The 9 is now home and will never move again.

That is the invariant: after pass i , the last i entries are the i largest values, in order.

Trace: All Five Passes

SORTING

pass	i	j range	comparisons	swaps	list after pass
—		—	—	—	7 2 1 4 5 9
1		1–5	5	4	2 1 4 5 7 9
2		1–4	4	1	1 2 4 5 7 9
3		1–3	3	0	1 2 4 5 7 9
4		1–2	2	0	1 2 4 5 7 9
5		1–1	1	0	1 2 4 5 7 9

$5+4+3+2+1 = 15$

The list was already sorted after pass 2. Passes 3 through 5 do six comparisons and change nothing.

Bubble sort as written always does $n(n-1)/2$ comparisons, no matter the input. Fixing that takes one boolean: stop early if a pass makes no swaps.

Bubble Sort, Formally

SORTING

```
BUBBLESORT( $a_1, \dots, a_n$ : reals,  $n \geq 2$ )
```

```
1: for  $i \leftarrow 1$  to  $n - 1$  do  
2:   for  $j \leftarrow 1$  to  $n - i$  do  
3:     if  $a_j > a_{j+1}$  then  
4:       swap  $a_j$  and  $a_{j+1}$   
5:     end if  
6:   end for  
7: end for
```

Two nested loops, and the inner bound is what fixes the cost: $(n-1) + (n-2) + \dots + 1$ comparisons.

The inner range is $n - i$, not n : after pass i , the last i entries are already in their final places.

Insertion Sort

THE IDEA

Grow a sorted section from the left. Each round, take the next element and slide it back into its correct place among the ones already sorted.

INPUT REQUIRED

STRUCTURE	Array $a_1 \dots a_n$ with $n \geq 2$, rearranged in place
ORDERING	None — but nearly-sorted input runs much faster
ELEMENTS	Real numbers — anything totally ordered
RETURNS	Nothing. The array itself is left sorted.

THINK OF IT AS

Sorting a hand of playing cards as they are dealt to you. Each new card slides left past the bigger ones until it sits where it belongs.

- 1 Treat the first entry on its own as a sorted section of length one.
- 2 Pick up the next entry and hold it aside. Call it the key.
- 3 Compare the key with the sorted entries, right to left. Slide every one that is larger than the key one place right.
- 4 Stop at the first entry not larger than the key, and drop the key into the gap you opened.
- 5 The sorted section is now one longer. Repeat until nothing is left to pick up.

Trace: Insertion Sort on 7 2 1 4 5 9

SORTING

j	key	shifted right	comps	list after
—	—	—	—	[7] 2 1 4 5 9
2	2	7	1	[2 7] 1 4 5 9
3	1	7, 2	2	[1 2 7] 4 5 9
4	4	7	2	[1 2 4 7] 5 9
5	5	7	2	[1 2 4 5 7] 9
6	9	nothing	1	[1 2 4 5 7 9]

8 comparisons

Brackets mark the sorted prefix. It grows by one element per round.

Eight comparisons against bubble sort’s **fifteen** , on the very same input.

Round 6 costs one comparison and no shifting, because 9 is already where it belongs.

Insertion Sort, Formally

SORTING

```
INSERTIONSORT( $a_1, \dots, a_n$ : reals,  $n \geq 2$ )
```

```
1: for  $j \leftarrow 2$  to  $n$  do  
2:    $key \leftarrow a_j$   
3:    $i \leftarrow j - 1$   
4:   while ( $i > 0$  and  $a_i > key$ ) do  
5:      $a_{i+1} \leftarrow a_i$   
6:      $i \leftarrow i - 1$   
7:   end while  
8:    $a_{i+1} \leftarrow key$   
9: end for
```

The `while` loop does the sliding; line 8 drops the key in.

Line 8 writes to a_{i+1} , not a_i , because the loop decremented i one step past the gap before exiting.

Same short-circuit trick on line 4 — test $i > 0$ before reading a_i .

Bubble vs Insertion

COUNTING

input, n = 6	bubble	insertion
7 2 1 4 5 9	15	8
1 2 4 5 7 9 already sorted	15	5
9 7 5 4 2 1 reversed	15	15

BUBBLE

Always $n(n-1)/2$ comparisons. The input never matters.

INSERTION

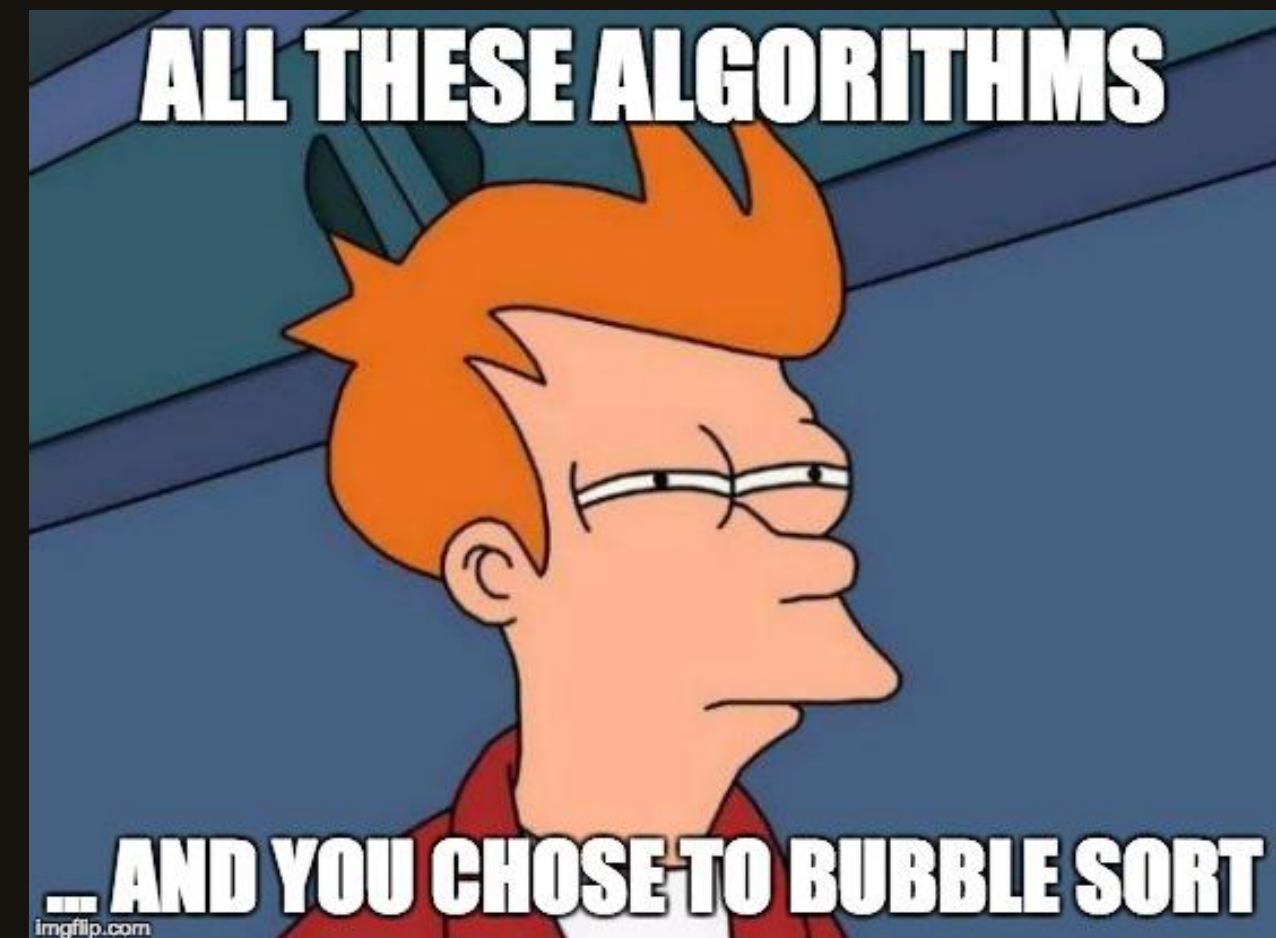
Between $n-1$ and $n(n-1)/2$. Nearly-sorted input is cheap.

Both are quadratic in the worst case, and both lose badly to the sorts you will meet in CSE 2331. What we need now is a language for saying “quadratic” precisely.

15 COMPARISONS, 6 ELEMENTS

Bubble sort: universally taught, never shipped.

It keeps comparing a sorted list because nobody told it to stop. Learn it, then never write it again.



What Each Algorithm Assumes

PRECONDITIONS

algorithm	data structure	ordering required	element type	returns
MAX	Array or list	none	Comparable (integers)	Largest value
LINEARSEARCH	Array, indexed from 1	none	Distinct integers	Position, or 0
BINARYSEARCH	Array, direct index access	sorted increasing	Distinct integers	Position, or 0
BUBBLESORT	Array, $n \geq 2$, in place	none	Reals (totally ordered)	Sorted array
INSERTIONSORT	Array, $n \geq 2$, in place	none	Reals (totally ordered)	Sorted array

Exactly one row makes a demand of the input, and it belongs to the only fast algorithm on the list. That is the trade: **binary search buys its speed with a precondition.**

Note the structure column too. Binary search needs to jump straight to a_m , so it needs an array. A perfectly sorted linked list still will not do — reaching the middle would cost a walk, and the halving would buy you nothing.

Common Mistakes

BEFORE YOU GO

- 01 Running binary search on an unsorted list. It returns an answer, and the answer is wrong.
- 02 Off-by-one on a loop bound. Bubble sort's inner loop ends at $n - i$, and it compares a_j with a_{j+1} — so j must never reach n .
- 03 Writing $j \leftarrow m - 1$ in binary search. The midpoint may still be the answer, so it is $j \leftarrow m$.
- 04 Reading pseudocode without tracing it. If you cannot run it by hand on six elements, you do not yet know what it does.

Recap

LECTURE 06

- 01 An algorithm is finite and precise. Pseudocode keeps it precise without arguing about syntax.
- 02 Finding the maximum costs $n - 1$ comparisons on every input.
- 03 Linear search needs no order and costs up to n . Binary search needs sorted input and costs $\lceil \log_2 n \rceil$.
- 04 Bubble sort always does $n(n-1)/2$ comparisons. Insertion sort does fewer when the input is nearly sorted.
- 05 Trace by hand on a small array. It is the fastest way to understand any procedure.

Next: naming these growth rates precisely — big O, big Omega, big Theta.